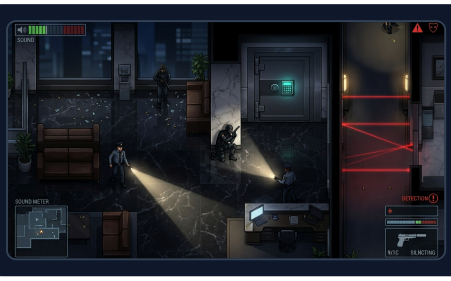
SOFTWARE ENGINEERING & CLEAN ARCHITECTURE SPECIFICATION

SHADOW HEIST (Raylib C#) — 4-Week Team Master Plan

A structured 4-sprint game engineering project designed to master **SOLID C# Architecture, Design Patterns, Unit Testing, and GitHub Pull Request Workflows** ahead of the Graduation Project.

Guindy: Core Engine, World Interactables & HUD | **Mahmoud:** Player Engine, Stances, Inventory & Gadgets

Hatem: Guard AI State Machine & A* Pathfinding | **Ali:** Level Loader, Vision Raycasting & 2D Lighting



Target Visual & Gameplay Direction: Gritty HD 2D Tactical Stealth

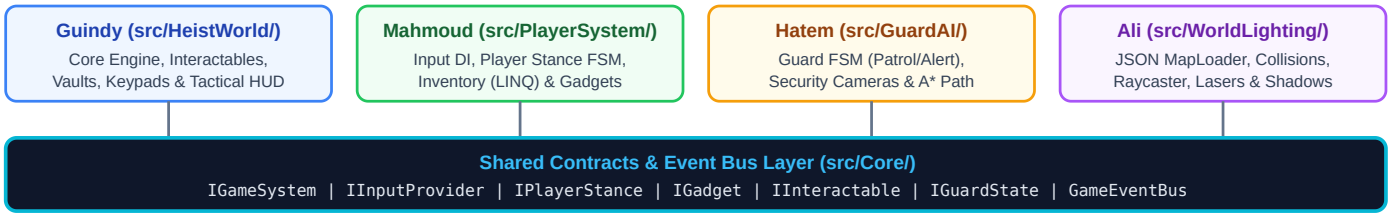
Inspired by *Intravenous 2* & *Door Kickers*. Top-down infiltration through dark marble banks and penthouses featuring real-time 2D shadow casting, flashlight vision cones, laser tripwires, keypad hacking, sound propagation meters, and smart patrolling guards.

`C# .NET 8 + Raylib-cs` `SOLID Principles` `State & Strategy Patterns` `Event Bus Decoupling` `xUnit Automated Tests`

1. Team Ownership & Subsystem Architecture

ZERO MERGE CONFLICT DESIGN

To prevent Git merge conflicts and enforce clean modular boundaries, each engineer owns **one dedicated C# namespace and directory**. Modules never reach into each other's private fields; they communicate strictly through **shared interfaces and the Event Bus** in `src/Core/`.



ENGINEER	OWNED NAMESPACE	CORE FEATURES & CLASSES	SOFTWARE ENGINEERING SKILLS PRACTICED
Guindy	`ShadowHeist.Core` `ShadowHeist.HeistWorld`	`GameContext`, `GameEventBus`, `VaultDoor`, `KeypadTerminal`, `LootCrate`, `FuseBox`, `HudRenderer`, `GameStateManager`	**Observer Pattern (Event Bus)**, **Polymorphism (`IInteractable`)**, Dependency Injection, High-Level State Machines
Mahmoud	`ShadowHeist.PlayerSystem`	`IInputProvider`, `Player`, `SneakStance`, `WalkStance`, `SprintStance`, `PlayerInventory`, `CoinGadget`, `EmpGadget`, `SuppressedPistol`	**Input Abstraction (DI)**, **State Pattern (`IPlayerStance`)**, **Strategy Pattern (`IGadget`)**, **C# Collections & LINQ**, **xUnit Testing**
Hatem	`ShadowHeist.GuardAI`	`Guard`, `PatrolState`, `SuspiciousState`, `AlertChaseState`, `SecurityCamera`, Detection Meter, `AStarPathfinder`	**Finite State Machine (`IGuardState`)**, **Graph Algorithms (2D A*)**, Event-Driven AI Transitions, **xUnit Testing**
Ali	`ShadowHeist.WorldLighting`	`MapLoader` (JSON), `CollisionHelper`, `VisionRaycaster`, `LaserTripwire`, `BrokenGlassTile`, `ShadowRenderer`, Bloom Shader	**Data-Driven Design (DTOs)**, **Computational Geometry (Raycasting)**, Custom Rendering Pipeline (`RenderTexture2D` & GLSL)

Why This Architecture Prepares Everyone for the Graduation Project: Every teammate works with **C# Interfaces, Dependency Injection, Event-Driven Decoupling, LINQ, and Automated Unit Tests**—the exact architectural building blocks used in enterprise backends and full-stack graduation projects.

2. Clean C# Solution Structure & SOLID Contracts

2.1 Repository Layout (`ShadowHeist.sln`)

```
ShadowHeist/
├── .github/
│   └── pull_request_template.md
├── assets/
│   ├── sprites/    # Characters, Tiles, Props
│   ├── shaders/    # 2d_lighting.fs, bloom.fs
│   ├── maps/       # bank_lvl1.json,
penthouse_lvl2.json
│   └── audio/      # sfx_step.wav, alarm.wav
├── src/
│   ├── Program.cs  # Locked at 40 lines (Entry Only)
│   ├── Core/       # Shared Interfaces & EventBus
│   │   ├── Interfaces/
│   │   │   ├── IGameSystem.cs
│   │   │   ├── IInputProvider.cs
│   │   │   ├── IPlayerStance.cs
│   │   │   ├── IInteractable.cs
│   │   │   ├── IGuardState.cs
│   │   │   └── IGadget.cs
│   │   ├── Events/
│   │   │   ├── GameEventBus.cs
│   │   │   ├── NoiseEmittedEvent.cs
│   │   │   └── AlarmTriggeredEvent.cs
│   │   └── GameContext.cs
│   ├── PlayerSystem/   # Owned by Mahmoud
│   ├── HeistWorld/     # Owned by Guindy
│   ├── GuardAI/        # Owned by Hatem
│   └── WorldLighting/  # Owned by Ali
└── tests/
    └── ShadowHeist.Tests/ # xUnit Test Project
```

2.2 The 4 Core Clean Code Rules

1. Single Responsibility & Input DI

Never mix `Raylib.IsKeyDown()` or drawing calls inside logic classes. **Mahmoud** reads input via `IInputProvider` and passes a `PlayerInputCommand` struct to `Player.Update()`, making logic 100% unit-testable.

2. State Pattern Over `if/else` Chains

No 400-line `switch` or nested `if/else` blocks. Both Player Stances (`IPlayerStance`) and Guard AI (`IGuardState`) encapsulate each state in its own small class (<100 lines).

3. Open/Closed Principle via Polymorphism

The Player interacts with any world object implementing `IInteractable` and uses any tool implementing `IGadget`. Adding a new gadget or door requires zero edits to `Player.cs`.

4. Event Bus for Cross-Module Messaging

When a player sprints or trips a laser, publish an event (`NoiseEmittedEvent` , `AlarmTriggeredEvent`) on `GameEventBus` so AI and HUD react without tight coupling.

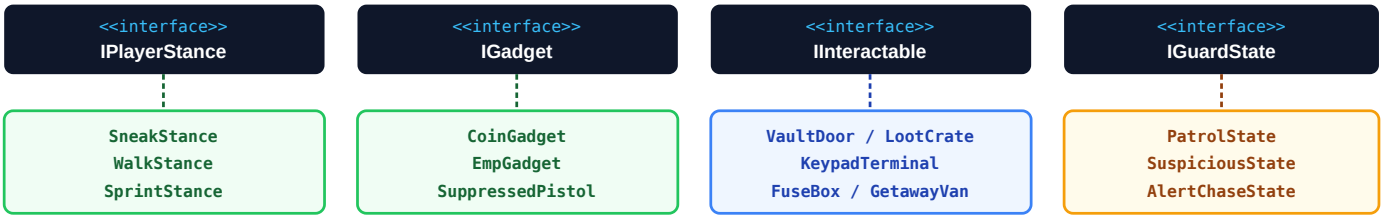
2.3 Core C# Interface Contracts (`src/Core/Interfaces/`)

```
// Implemented by SneakStance, WalkStance,
SprintStance
public interface IPlayerStance
{
    string Name { get; }
    float SpeedMultiplier { get; }
    float NoiseRadius { get; }
    IPlayerStance HandleInput(PlayerInputCommand
cmd);
}

// Implemented by CoinGadget, EmpGadget,
SuppressedPistol
public interface IGadget
{
    string Name { get; }
    int RemainingCharges { get; }
    bool TryUse(Vector2 origin, Vector2 target,
GameEventBus bus);
}
```

```
// Implemented by VaultDoor, KeypadTerminal,
LootCrate, FuseBox
public interface IInteractable
{
    Vector2 Position { get; }
    float InteractRadius { get; }
    string PromptText { get; }
    bool CanInteract(Player player);
    void OnInteract(Player player, GameEventBus bus);
}

// Implemented by PatrolState, SuspiciousState,
AlertChaseState
public interface IGuardState
{
    void Enter(Guard guard);
    IGuardState Update(Guard guard, float dt,
GameContext ctx);
}
```



3. GitHub Team Workflow & Peer Code Review Playbook

GIT STANDARDS

To build strong habits for the Graduation Project, **direct pushes to `main` are disabled**. Every feature is developed on an isolated branch, verified to build cleanly, and merged via a Pull Request reviewed by two teammates.

3.1 Branch Naming & Commit Convention

Every branch follows the pattern `<type>/<owner>-<feature>`:

```
feat/guindy-keypad-terminal
feat/mahmoud-player-stances
feat/mahmoud-inventory-linq
feat/hatem-guard-patrol-state
feat/ali-vision-raycaster
fix/ali-wall-corner-collision
```

Conventional Commit Messages:

```
feat(player): add SneakStance and SprintStance
feat(ai): implement 2D grid AStarPathfinder
fix(lighting): clip vision ray at wall edges
test(inventory): add xUnit tests for loot weight
```

3.2 The Daily 5-Step Git Routine

Run these exact commands before starting any new ticket:

```
# Step 1: Switch to main & pull latest team code
git checkout main
git pull origin main

# Step 2: Create a fresh branch for ONE ticket
git checkout -b feat/mahmoud-coin-gadget

# Step 3: Verify build & tests pass locally
dotnet build && dotnet test

# Step 4: Stage & commit with clean message
git add .
git commit -m "feat(gadgets): add CoinGadget"

# Step 5: Push & open Pull Request on GitHub
git push -u origin feat/mahmoud-coin-gadget
```

3.3 Mandatory Peer Review Rotation Matrix (Everyone Reviews Code Every Week)

Reviewing teammates' Pull Requests is one of the fastest ways to learn clean C#. Every Pull Request requires **2 Approvals** before merging:

PR AUTHOR	PRIMARY REVIEWER	SECONDARY REVIEWER	WHAT REVIEWERS FOCUS ON
Mahmoud	Hatem	Guindy	Verify `IInputProvider` abstraction, clean State/Strategy classes, and zero hardcoded magic numbers.
Hatem	Mahmoud	Ali	Read through Guard FSM state transitions & A* pathfinding; check method clarity and naming.
Ali	Mahmoud	Guindy	Check JSON DTO models, raycast math helper methods, and proper shader/texture disposal.
Guindy	Hatem	Ali	Verify `Core` interfaces, EventBus contracts, and `IInteractable` decoupling.

3.4 Pull Request Review Checklist (`.github/pull_request_template.md`)

- [ ] **1. Compiles & Passes Tests:** `dotnet build` and `dotnet test` succeed with 0 errors.
- [ ] **2. Single Responsibility (SRP):** Every new class is under ~150 lines and has one clear responsibility (logic vs. rendering separated).
- [ ] **3. No Magic Numbers:** Speeds, radii, timers, and dimensions use named constants (e.g., `private const float MaxVisionDist = 240f;`).
- [ ] **4. C# Naming Conventions:** Classes, methods, and properties use `PascalCase` ; private fields use `_camelCase` ; interfaces start with `I` .
- [ ] **5. Strict Module Boundaries:** Only files inside the author's assigned folder were modified (unless agreed in a Core contract update).

4. 4-Week Sprint Plan — Weeks 1 & 2

TICKETS #1 – #16

🗓️ Week 1: Core Skeleton, Input Abstraction, Map Loading & First Patrols

Sprint 1 Goal: Practice the complete Git branch → Pull Request → Peer Review workflow while getting the player moving, guards patrolling, and JSON bank map rendering.

ID	OWNER	BRANCH NAME	DELIVERABLE & C# CLASSES	CLEAN CODE FOCUS
#1	Guindy	`feat/guindy-core-skeleton`	Initialize solution, folders, `GameContext`, `GameEventBus`, and core interfaces in `src/Core/`.	Interfaces & Contracts
#2	Guindy	`feat/guindy-loot-crate`	Implement `LootCrate.cs : IInteractable` & initial `HudRenderer.cs` showing stolen cash total.	Polymorphism (`IInteractable`)
#3	Mahmoud	`feat/mahmoud-input-provider`	Create `IInputProvider`, `KeyboardInputProvider.cs`, and immutable `PlayerInputCommand` struct.	Dependency Injection & SRP
#4	Mahmoud	`feat/mahmoud-player-movement`	Implement `Player.cs` movement & mouse-aim rotation cleanly separated from `PlayerRenderer.cs`.	Logic vs. View Separation
#5	Hatem	`feat/hatem-guard-patrol`	Create `Guard.cs` entity and `PatrolState.cs : IGuardState` cycling through waypoint coordinates.	State Pattern Foundation
#6	Hatem	`feat/hatem-guard-rotation`	Add smooth angular interpolation toward target waypoint + debug patrol line gizmos.	Encapsulated Math Helpers
#7	Ali	`feat/ali-json-map-loader`	Build `MapLoader.cs` deserializing walls, marble floor tiles, and spawn points from JSON.	Data-Driven DTOs
#8	Ali	`feat/ali-wall-collision`	Implement `CollisionHelper.ResolveWallCollision()` for smooth circle-vs-AABB wall sliding.	Pure Static Functions

🗓️ Week 2: State Machines, Event Bus, Vision Raycasting & Keypad Hacking

Sprint 2 Goal: Replace `if/else` flags with the **State Pattern** (`IPlayerStance`, `IGuardState`) and connect player noise, guard investigation, and laser alarms via `GameEventBus`.

ID	OWNER	BRANCH NAME	DELIVERABLE & C# CLASSES	CLEAN CODE FOCUS
#9	Guindy	`feat/guindy-vault-keypad`	Create `VaultDoor.cs` and `KeypadTerminal.cs : IInteractable` (hold `E` to fill hacking bar & unlock).	Object Composition
#10	Guindy	`feat/guindy-sound-meter-hud`	Add dynamic **Sound Meter Bar** & **Hacking Progress Ring** to `HudRenderer.cs` via EventBus.	Observer Pattern in UI
#11	Mahmoud	`feat/mahmoud-stance-fsm`	Implement `IPlayerStance` with 3 classes: `SneakStance.cs`, `WalkStance.cs`, and `SprintStance.cs`.	State Pattern & Open/Closed
#12	Mahmoud	`feat/mahmoud-noise-emitter`	Publish `NoiseEmittedEvent` from active stance when moving (silent crouch vs. loud sprint ripples).	Event-Driven Publishing
#13	Hatem	`feat/hatem-suspicious-state`	Implement `SuspiciousState.cs`: guard hears `NoiseEmittedEvent`, pauses, and walks to investigate.	Decoupled AI Listeners
#14	Hatem	`feat/hatem-astar-pathfinder`	Implement 2D grid `AStarPathfinder.cs` so investigating guards navigate cleanly around walls.	Algorithm Encapsulation
#15	Ali	`feat/ali-vision-raycaster`	Build `VisionRaycaster.cs` computing a guard's field-of-view triangle fan clipped by walls.	Ray-Segment Intersection
#16	Ali	`feat/ali-laser-tripwire`	Create `LaserTripwire.cs` rendering red laser beams that trigger `AlarmTriggeredEvent` on contact.	Raycasting & Events

5. 4-Week Sprint Plan — Weeks 3 & 4

TICKETS #17 – #32

🗓️ Week 3: Inventory/LINQ, Heist Gadgets, Alert Chase & 2D Shadow Casting

Sprint 3 Goal: Build `PlayerInventory` with C# Collections & LINQ, add polymorphic Heist Gadgets (`IGadget`), complete the Guard Alert Chase loop, and add 2D dynamic shadows.

ID	OWNER	BRANCH NAME	DELIVERABLE & C# CLASSES	CLEAN CODE FOCUS
#17	Guindy	`feat/guindy-fusebox-switch`	Create `FuseBox.cs` & `LaserSwitch.cs : IInteractable` to disable linked room lights and laser grids.	Loose Coupling via IDs
#18	Guindy	`feat/guindy-camera-peek`	Implement `TacticalCamera2D.cs` with smooth lerp follow, alarm screen-shake, and Right-Click peek.	Encapsulated Camera Logic
#19	Mahmoud	`feat/mahmoud-inventory-linq`	Build `PlayerInventory.cs` using `List<LootItem>` , `Dictionary` , and **LINQ** (`Sum` for weight slowdown).	C# Collections & LINQ
#20	Mahmoud	`feat/mahmoud-heist-gadgets`	Implement `IGadget` Strategy Pattern: `CoinGadget.cs` (distraction) & `EmpGadget.cs` (jam cameras).	Strategy Pattern (`IGadget`)
#21	Hatem	`feat/hatem-alert-chase-state`	Implement `AlertChaseState.cs` : guard spots player, triggers bank alarm, and chases last seen position.	Complete Guard FSM
#22	Hatem	`feat/hatem-detection-meter`	Add progressive Detection Meter (0% → 100% fill before instant alarm, scaling with player distance).	Clean State Timers
#23	Ali	`feat/ali-shadow-renderer`	Implement `ShadowRenderer.cs` using `RenderTexture2D` darkness mask + flashlight vision cutouts.	Multi-Pass Render Pipeline
#24	Ali	`feat/ali-glass-hazards`	Add `BrokenGlassTile` surface hazard (spikes footstep noise) + emissive glow sprites on keypads.	Layered Render Sorting

🗓️ Week 4: Automated Unit Testing (`xUnit`), Security Cameras, Menus & Polish

Sprint 4 Goal: Write automated `xUnit` tests for decoupled logic classes, add Level 2, complete the Mission Victory/Busted loop, and prepare the Graduation Readiness Retrospective.

ID	OWNER	BRANCH NAME	DELIVERABLE & C# CLASSES	CLEAN CODE FOCUS
#25	Guindy	`feat/guindy-game-loop-states`	Build `GameStateManager` (`MainMenu` → `Playing` → `Victory` / `Busted`) + `GetawayVanZone` .	High-Level State Machine
#26	Guindy	`feat/guindy-summary-ui-docs`	Create Mission Summary Screen (Stealth Rank `S/A/B`) + write final Architecture `README.md` .	Technical Documentation
#27	Mahmoud	`feat/mahmoud-suppressed-pistol`	Add `SuppressedPistol.cs : IGadget` (limited subsonic rounds to shoot out lights or stun guards).	Extending `IGadget` Cleanly
#28	Mahmoud	`test/mahmoud-player-unit-tests`	Write **5 `xUnit` Tests** verifying `PlayerInventory` weight math, stance transitions, and gadget charges.	Automated Unit Testing
#29	Hatem	`feat/hatem-security-camera`	Implement sweeping `SecurityCamera.cs` that alerts nearest guard when player enters its cone.	Component Reuse
#30	Hatem	`test/hatem-ai-unit-tests`	Write **5 `xUnit` Tests** verifying `AStarPathfinder` shortest paths and Guard FSM transitions.	Automated Unit Testing
#31	Ali	`feat/ali-levels-and-sprites`	Design **Level 1 (City Bank)** & **Level 2 (Luxury Penthouse)** JSON maps + integrate HD sprite sheets.	Content Pipeline
#32	Ali	`feat/ali-bloom-and-audio`	Add GLSL Bloom shader for lasers/lights and `AudioManager : IDisposable` for clean sound management.	`IDisposable` Resource Safety

> **Friday 30-Minute Merge & Playtest Protocol:** Every Friday, the team hops on a call, reviews any remaining open PRs together, merges to `main` , runs `git pull origin main` on all 4 laptops, and playtests the new heist features together before starting the next week's sprint!